ОТЧЁТ

по лабораторной работе №14

«Разработка REST API на FastAPI»

Вариант 20. Интернет-провайдер — тарифы

1. Цель работы

Научиться публиковать ресурс /tariffs на FastAPI: модели Pydantic, коды ответов, фильтрация, сортировка, поиск по скорости и тесты через TestClient.

2. Постановка индивидуального задания

Вариант 20. Поля тарифа: название, скорость, цена, трафик, услуги, регион и статус. Адрес ресурса — /tariffs. Нужны список, объект, создание, замена, частичное изменение, удаление и выборка по скорости.

3. Краткие теоретические сведения

FastAPI описывает тело запроса моделью Pydantic. Неверный тип или пустое название отклоняются до входа в функцию и дают 422. Параметры пути и query объявляются в аргументах функции. Декоратор задаёт метод и адрес.

OpenAPI-схема строится по этим аннотациям. Клиент может быть отдельной программой: в работе это консоль на httpx, тесты ходят через TestClient без настоящего порта.

4. Описание программы

Стартовый набор — три тарифа. GET /tariffs фильтрует по статусу и региону и сортирует по цене или скорости. GET /tariffs/by-speed — операция варианта. POST проверяет статус «активен» или «архив». PUT заменяет документ, PATCH меняет только переданные поля, DELETE отвечает 204.

Тесты проверяют список, порог скорости, создание с последующим PATCH, отсутствующий id, удаление и отказ невалидного тела.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — app.py

```
"""
Лабораторная работа №14. Вариант 20.
REST API тарифов на FastAPI.
"""

from fastapi import FastAPI, HTTPException, Query
```

```
from pydantic import BaseModel, Field


class TariffIn(BaseModel):
    name: str = Field(min_length=1)
    speed: int = Field(gt=0)
    price: float = Field(ge=0)
    traffic: str = "безлимит"
    services: list[str] = []
    region: str = "Москва"
    status: str = "активен"


class Tariff(TariffIn):
    id: int


class TariffPatch(BaseModel):
    name: str | None = None
    speed: int | None = Field(default=None, gt=0)
    price: float | None = Field(default=None, ge=0)
    traffic: str | None = None
    services: list[str] | None = None
    region: str | None = None
    status: str | None = None


def create_app():
    app = FastAPI(title="Тарифы провайдера")
    items: dict[int, dict] = {}
    seq = {"n": 1}

    def add(data: dict) -> dict:
        item = dict(data)
        item["id"] = seq["n"]
        seq["n"] += 1
        items[item["id"]] = item
        return item

    add({"name": "Домашний 100", "speed": 100, "price": 500, "traffic": "безлимит",
         "services": ["антивирус"], "region": "Москва", "status": "активен"})
    add({"name": "Домашний 300", "speed": 300, "price": 800, "traffic": "безлимит",
         "services": ["ТВ"], "region": "Казань", "status": "активен"})
    add({"name": "Корпоративный 500", "speed": 500, "price": 2500, "traffic": "гарантия",
         "services": ["белый IP"], "region": "Москва", "status": "активен"})

    @app.get("/tariffs")
    def list_tariffs(
        status: str | None = None,
        region: str | None = None,
        sort: str | None = None,
    ):
        result = list(items.values())
        if status:
            result = [item for item in result if item["status"] == status]
        if region:
            result = [item for item in result if item["region"] == region]
        if sort == "price":
            result = sorted(result, key=lambda item: item["price"])
        elif sort == "speed":
            result = sorted(result, key=lambda item: item["speed"], reverse=True)
        return result

    @app.get("/tariffs/by-speed")
    def by_speed(min_speed: int = Query(gt=0)):
        return [item for item in items.values() if item["speed"] >= min_speed]

    @app.get("/tariffs/{item_id}")
    def get_tariff(item_id: int):
        item = items.get(item_id)
        if item is None:
            raise HTTPException(404, "тариф не найден")
```

```
        return item

    @app.post("/tariffs", status_code=201)
    def create_tariff(payload: TariffIn):
        if payload.status not in {"активен", "архив"}:
            raise HTTPException(400, "статус: активен или архив")
        return add(payload.model_dump())

    @app.put("/tariffs/{item_id}")
    def replace_tariff(item_id: int, payload: TariffIn):
        if item_id not in items:
            raise HTTPException(404, "тариф не найден")
        saved = payload.model_dump()
        saved["id"] = item_id
        items[item_id] = saved
        return saved

    @app.patch("/tariffs/{item_id}")
    def patch_tariff(item_id: int, payload: TariffPatch):
        item = items.get(item_id)
        if item is None:
            raise HTTPException(404, "тариф не найден")
        changes = payload.model_dump(exclude_unset=True)
        item.update(changes)
        return item

    @app.delete("/tariffs/{item_id}", status_code=204)
    def delete_tariff(item_id: int):
        if item_id not in items:
            raise HTTPException(404, "тариф не найден")
        del items[item_id]

    return app


app = create_app()

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=5014)
```

Листинг — client.py

```
"""Консольный клиент FastAPI тарифов."""

import json
import sys

import httpx

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")

BASE = "http://127.0.0.1:5014"


def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()


def show(response: httpx.Response):
    print(f"HTTP {response.status_code}")
```

```
    if response.content:
        print(json.dumps(response.json(), ensure_ascii=False, indent=2))


def main():
    try:
        httpx.get(BASE + "/tariffs", timeout=3)
    except httpx.HTTPError as exc:
        print("Сервер не запущен. Команда: python -m uvicorn app:app --port 5014")
        print(exc)
        return
    client = httpx.Client(base_url=BASE, timeout=5)
    actions = {
        "1": lambda: show(client.get("/tariffs")),
        "2": lambda: show(client.get(f"/tariffs/{read_line('ID: ')}")),
        "3": lambda: show(client.post("/tariffs", json={
            "name": read_line("Название: "),
            "speed": int(read_line("Скорость: ")),
            "price": float(read_line("Цена: ")),
            "traffic": read_line("Трафик: ") or "безлимит",
            "services": ["антивирус"],
            "region": read_line("Регион: ") or "Москва",
            "status": "активен",
        })),
        "4": lambda: show(client.patch(f"/tariffs/{read_line('ID: ')}", json={
            "price": float(read_line("Новая цена: ")),
        })),
        "5": lambda: show(client.get("/tariffs/by-speed", params={"min_speed": read_line("Мин. скорость:
")})),
        "6": lambda: show(client.get("/tariffs", params={"sort": "price"})),
        "7": lambda: show(client.delete(f"/tariffs/{read_line('ID: ')}")),
    }
    while True:
        print()
        print("===== FastAPI /tariffs =====")
        print("1. Список  2. Объект  3. Создать  4. Изменить цену")
        print("5. По скорости  6. Сортировка по цене  7. Удалить  0. Выход")
        choice = read_line("Выберите действие: ")
        if choice == "0":
            print("Работа клиента завершена.")
            break
        action = actions.get(choice)
        if action is None:
            print("Ошибка: неизвестный пункт.")
            continue
        try:
            action()
        except (httpx.HTTPError, ValueError) as exc:
            print(f"Ошибка: {exc}")
        except EOFError:
            break


if __name__ == "__main__":
    main()
```

Листинг — tests\test_api.py

```
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from fastapi.testclient import TestClient

from app import create_app


def api():
    return TestClient(create_app())


def test_list():
    response = api().get("/tariffs")
```

```
    assert response.status_code == 200
    assert len(response.json()) == 3


def test_by_speed():
    response = api().get("/tariffs/by-speed", params={"min_speed": 300})
    assert response.status_code == 200
    assert all(item["speed"] >= 300 for item in response.json())


def test_create_and_patch():
    client = api()
    created = client.post("/tariffs", json={
        "name": "Ночной", "speed": 50, "price": 300, "traffic": "ночь",
        "services": [], "region": "Тула", "status": "активен",
    })
    assert created.status_code == 201
    item_id = created.json()["id"]
    patched = client.patch(f"/tariffs/{item_id}", json={"price": 280, "status": "архив"})
    assert patched.status_code == 200
    assert patched.json()["price"] == 280


def test_missing():
    assert api().get("/tariffs/99").status_code == 404


def test_delete():
    client = api()
    assert client.delete("/tariffs/1").status_code == 204
    assert client.get("/tariffs/1").status_code == 404


def test_validation():
    response = api().post("/tariffs", json={"name": "", "speed": 0, "price": -1})
    assert response.status_code == 422
```

6. Примеры работы программы

Снимки клиента показывают список и поиск по скорости, создание тарифа и сортировку, а также тесты.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python14> python client.py


===== FastAPI /tariffs =====
1. Список  2. Объект  3. Создать  4. Изменить цену
5. По скорости  6. Сортировка по цене  7. Удалить  0. Выход
Выберите действие: 1
HTTP 200
[
  {
    "name": "Домашний 100",
    "speed": 100,
    "price": 500,
    "traffic": "безлимит",
    "services": [
      "антивирус"
    ],
    "region": "Москва",
    "status": "активен",
    "id": 1
  },
  {
    "name": "Домашний 300",
    "speed": 300,
    "price": 800,
    "traffic": "безлимит",
    "services": [
      "ТВ"
    ],
    "region": "Казань",
    "status": "активен",
    "id": 2
  },
  {
    "name": "Корпоративный 500",
    "speed": 500,
    "price": 2500,
    "traffic": "гарантия",
    "services": [
      "белый IP"
    ],
    "region": "Москва",
    "status": "активен",
    "id": 3
  },
  {
    "name": "Ночной",
    "speed": 50,
    "price": 700.0,
    "traffic": "ночь",
```

Рисунок 1 – Список тарифов и выборка по скорости

```
C:\\Users\\Shmurdyakich\\Desktop\\pythonALL\\python14> python -m pytest -q

......                                                                   [100%]
============================== warnings summary ===============================
..\..\..\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python
312\site-packages\fastapi\testclient.py:1
  C:\Users\Shmurdyakich\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-
packages\Python312\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlett
e.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
6 passed, 1 warning in 0.37s
```

Рисунок 2 – Тесты FastAPI

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python14> python client.py


===== FastAPI /tariffs =====
1. Список  2. Объект  3. Создать  4. Изменить цену
5. По скорости  6. Сортировка по цене  7. Удалить  0. Выход
Выберите действие: 3
Название: Ночной
Скорость: 50
Цена: 300
Трафик: ночь
Регион: Тула
HTTP 201
{
  "name": "Ночной",
  "speed": 50,
  "price": 300.0,
  "traffic": "ночь",
  "services": [
    "антивирус"
  ],
  "region": "Тула",
  "status": "активен",
  "id": 5
}

===== FastAPI /tariffs =====
1. Список  2. Объект  3. Создать  4. Изменить цену
5. По скорости  6. Сортировка по цене  7. Удалить  0. Выход
Выберите действие: 6
HTTP 200
[
  {
    "name": "Ночной",
    "speed": 50,
    "price": 300.0,
    "traffic": "ночь",
    "services": [
      "антивирус"
    ],
    "region": "Тула",
    "status": "активен",
    "id": 5
  },
  {
    "name": "Домашний 100",
    "speed": 100,
    "price": 500,
    "traffic": "безлимит",
    "services": [
```

Рисунок 3 – Создание тарифа и сортировка по цене

7. Ответы на контрольные вопросы

1. Что проверяет Pydantic до функции?

Типы и ограничения полей: скорость больше нуля, цена не отрицательная, название не пустое. Ошибка приходит как 422.

2. Чем FastAPI отличается от ручного разбора JSON?

Схема запроса объявлена классом. Фреймворк сам читает тело и формирует описание API.

3. Какой адрес у специализированной операции?

GET /tariffs/by-speed?min_speed=... Он объявлен раньше адреса с id, чтобы слово by-speed не считалось номером.

4. Почему тесты не поднимают uvicorn?

TestClient вызывает приложение в процессе. Так проверяется контракт без отдельного сервера.

5. Что возвращает удаление?

Код 204 и пустое тело. Повторное чтение этого id даёт 404.

8. Вывод

В лабораторной работе №14 тарифы варианта 20 доступны через FastAPI по адресу /tariffs. Модели проверяют вход, выборка по скорости реализована отдельным адресом, клиент и тесты подтверждают создание, изменение, сортировку и удаление.